

ทีม LUMEN · เอกสารภายใน

คู่มือทำความเข้าใจระบบ JOD tang (จัดตั้ง)

ตั้งแต่เอกสาร ไปจนถึงโค้ด — ระบบทำงานอย่างไร และทำขึ้นมาเพื่ออะไร

Intro to Software Engineering (15031001)

ปรับปรุง 8 ตุลาคม 2026

github.com/6931503045-design/Lumen_line

สารบัญ

0. คู่มือนี้มีไว้ทำอะไร และอ่านอย่างไร

1. ระบบนี้ทำขึ้นมาเพื่ออะไร

- 1.1 ปัญหาที่เห็นจริง
- 1.2 สองคำถามที่ระบบนี้ตั้งใจจะตอบ
- 1.3 ทางแก้ของเรา — 3 เสา
- 1.4 ทำไมเลือก LINE ไม่ใช่แอปของตัวเอง
- 1.5 ขอบเขต — อะไรอยู่ใน อะไรอยู่นอก

2. แผนที่เอกสาร — ไฟล์ไหนคืออะไร

3. กฎเหล็ก 7 ข้อ — หัวใจของโปรเจกต์

- 3.1 ทำไมกฎแต่ละข้อถึงจำเป็น — คำตอบสำหรับ "ทำไมไม่ให้ AI คิดเลขไปเลย"

4. ภาพรวมสถาปัตยกรรม

- 4.1 ภาพใหญ่
- 4.2 เทคโนโลยีที่ใช้ และเหตุผลที่เลือก
- 4.3 เส้นทางที่ข้อมูลเข้าระบบ
- 4.4 Router 4 ชั้น — หัวใจของการ "จัดง่าย"

5. ไลโค็ดที่ละชั้น

- 5.1 หลักการจัดชั้น (layering) — กฎเดียวที่ต้องจำ
- 5.2 `src/config/` — ค่าคงที่และการตั้งค่า
- 5.3 `src/utils/` — ฟังก์ชันบริสุทธ์ (ชั้นล่างสุด)
- 5.4 `src/db/queries/` — ชั้นคุย Supabase
- 5.5 `src/services/` — business logic ทั้งหมด
- 5.6 S10 Dedup — มีแค่ 2 เงื่อนไข และห้ามเพิ่ม
- 5.7 `src/handlers/` — รับ event จาก LINE
- 5.8 `src/routes/` — ชั้น HTTP
- 5.9 ทางเดินของข้อความหนึ่งข้อความ (ตัวอย่างจริง)

6. Money Engine (S5) — เขตห้าม AI

- 6.1 Safe-to-spend — "ใช้ได้อีกวันละเท่าไร" (ตอบคำถามที่ 1)
- 6.2 Plan Capacity — "เก็บได้เดือนละเท่าไร" (ตอบคำถามที่ 2)
- 6.3 สามทางเลือก — เร็ว / สมดุล / สบาย
- 6.4 เหตุผลที่ระบบปฏิเสธการสร้างแผน — ต้องเป็นตัวเลขจับต้องได้
- 6.5 งบรายหมวด + เตือน 80%/100%

7. ชั้น AI — ให้ AI ช่วยได้ แต่ไม่ให้แตะเงิน

- 7.1 โครงสร้าง 6 ไฟล์
- 7.2 Guard 5 ด้าน — ลำดับสลับไม่ได้

7.3 8 Tools ที่ AI เลือกได้

7.4 สามด้านที่บังคับ G1 — AI ไม่มีทางแตะตัวเลขเงิน

7.5 วงจรอ่านสลิป (FR-14 ใน PDF / FR-15 ในฉบับไทย)

7.6 Pending Actions — ที่เดียวที่ AI กลายเป็นข้อมูลจริง

7.7 PDPA — การแจ้งผู้ใช้

8. ฐานข้อมูล — 12 ตาราง

8.1 ตาราง `transactions` — ข้อจำกัดที่ DB บังคับให้

8.2 RLS — ความปลอดภัยที่ต้องอธิบายให้ถูก

9. หน้าเว็บ

9.1 6 หน้า

9.2 ทำไมไม่มี `boot.js` — คำถามที่กรรมการนำถาม

9.3 โหมด preview — ทำไมต้องมี

9.4 Session — ล็อกอินด้วย LINE Login

10. งานตามเวลา (cron)

11. การทดสอบ — 683 ข้อ / 32 ไฟล์

11.1 หลักการ

11.2 ทดสอบทั้งหมด

11.3 แนวการเขียนเทสที่ควรเล่า

11.4 บั๊กที่เทสจับได้จริง (ตัวอย่างที่เล่าได้)

12. Deploy — ระบบจริงทำงานอยู่ที่ไหน

12.1 ตัวแปรสภาพแวดล้อมที่จำเป็น

12.2 รันในเครื่อง

13. คำถามที่กรรมการนำจะถาม + คำตอบ

Q1. "ระบบนี้ทำอะไร อธิบายใน 30 วินาที"

Q2. "ทำไมไม่ให้ AI คำนวณไปเลย จะง่ายกว่าไหม"

Q3. "AI เข้าใจผิดแล้วบันทึกเงินผิดจะอย่างไร"

Q4. "ถ้า AI ล่มหรือโศกนาฏกรรม แอปใช้ได้ไหม"

Q5. "ข้อมูลของผู้ใช้คนหนึ่งจะหลุดไปหาคนอื่นได้ไหม"

Q6. "เรื่อง PDPA จัดการยังไง ส่งข้อมูลไปให้ Google ด้วยไหม"

Q7. "ทำไมเงินเป็นสตางค์ ไม่เป็นบาททศนิยม"

Q8. "ทราบกับเพื่อนอย่างไรให้เศษไม่หาย"

Q9. "เจอปัญหาอะไรที่ไม่คาดคิดบ้าง" ← คำถามนี้ถามบ่อย ตอบให้ดี

Q10. "AI เขียนโค้ดให้เท่าไร ส่วนไหนเป็นของทีม"

Q11. "มีอะไรที่ยังทำไม่เสร็จ"

Q12. "ทำไมต้องมี 4 ชั้นใน router ชับซ้อนไปไหม"

Q13. "ถ้ามีเวลาอีก 1 เดือน จะทำอะไร"

14. ตัวเลขและศัพท์ที่ต้องจำได้

14.1 ค่าคงที่ (`src/config/constants.ts`)

14.2 ตัวเลขของโปรเจกต์

14.3 ศัพท์ที่ใช้ในทีม

15. ข้อจำกัดที่เรารู้ตัว — ตอบตรง ๆ ดีกว่าถูกจับได้

16. เช็กลิสต์ก่อนเข้าห้องสอบ

คู่มือทำความเข้าใจระบบ JOD tang (จัดตั้ง) ทั้งระบบ

ทีม Lumen · Intro to Software Engineering (15031001) · ปรับปรุง 8 ตุลาคม 2026 Repo: https://github.com/6931503045-design/Lumen_line · Demo: <https://lumen-line.onrender.com>

0. คู่มือนี้มีไว้ทำอะไร และอ่านอย่างไร

เกณฑ์ให้คะแนนของวิชานี้เขียนไว้ตรง ๆ ว่า

"Work you cannot explain earns no credit" — งานที่อธิบายไม่ได้ ไม่ได้คะแนน

เอกสารนี้จึงไม่ใช่เอกสารส่ง แต่เป็น เอกสารให้เราอ่านเอง เพื่อให้ทุกคนในทีม อธิบายระบบได้ ไม่ว่าจะเป็นส่วนที่ตัวเองเขียนหรือไม่

ถ้าคุณมีเวลา	อ่านส่วน
10 นาที (ก่อนเข้าห้องสอบ)	§1 · §3 · §13
1 ชั่วโมง (ซ้อมอธิบาย)	§1 → §7 แล้วข้ามไป §13
ครึ่งวัน (เข้าใจทั้งระบบ)	อ่านเรียงทั้งหมด

วิธีใช้ที่ได้ผลที่สุด: จับคู่กันถาม-ตอบจาก §13 แล้วถ้าตอบไม่ได้ให้ย้อนไปอ่านส่วนที่เกี่ยวข้อง

1. ระบบนี้ทำขึ้นมาเพื่ออะไร

1.1 ปัญหาที่เห็นจริง

เราเริ่มจากข้อสังเกต 3 ข้อเกี่ยวกับนักศึกษากับการจ่ายเงิน:

- หนึ่ง — จัดแล้วเล็ก แอปจดยรายจ่ายทั่วไปต้องกรอกหลายชั้น เปิดแอป → เลือกหมวด → พิมพ์ยอด → กดบันทึก คนส่วนใหญ่เล็ก จัดภายใน 2-3 สัปดาห์ ไม่ใช่เพราะขี้เกียจ แต่เพราะ **ต้นทุนต่อครั้งสูงเกินกว่าประโยชน์ที่เห็นทันที**
- สอง — รู้แค่ว่าเงินหายไปไหน แอปบอกได้ว่าเดือนนี้ใช้ไปเท่าไร แยกเป็นหมวดอะไร แต่ไม่ตอบคำถามที่คนอยากรู้จริง ๆ ว่า *"แล้วมันควรทำอะไรต่อ"*
- สาม — เงินออมเป็นเรื่องของความรู้สึก คนตั้งเป้าว่าจะเก็บเงินซื้อของ แต่ไม่รู้ว่า เดือนละเท่าไรจึงจะไหว และไม่รู้ว่าตอนนี้ยังไหวอยู่ไหม

1.2 สองคำถามที่ระบบนี้ตั้งใจจะตอบ

ทั้งโปรเจกต์ถูกออกแบบรอบคำถามสองข้อนี้ ถ้าพีเจอาร์ไหนไม่ช่วยตอบสองข้อนี้ มันคือ Could ไม่ใช่ Must

คำถามที่ 1 — "วันนี้ฉันใช้เงินได้อีกเท่าไร?" ไม่ใช่ "เดือนนี้ใช้ไปแล้วเท่าไร" ซึ่งเป็นอดีต แต่เป็นตัวเลขที่ใช้ตัดสินใจได้เดี๋ยวนี้

คำถามที่ 2 — "ฉันจะเก็บเงินให้ถึงเป้าได้อย่างไร?" ไม่ใช่ "ควรเก็บเงิน" ซึ่งใครก็พูดได้ แต่เป็นยอดต่อเดือนที่คำนวณจากพฤติกรรมจริงของเขา

1.3 ทางแก้ของเรา — 3 เสา

เสา	สิ่งที่ทำ	แก้ปัญหาข้อไหน
A — จัดง่ายที่สุดเท่าที่ทำได้	จดผ่าน LINE ที่เปิดอยู่แล้ว พิมพ์ กาแฟ 80 จบ · อีเมลธนาคารเข้าระบบเอง · ส่งรูปสลิปให้อ่าน	1 (จดแล้วเลิก)
B — ตอบคำถามไม่ใช่โชว์ตัวเลข	"ใช้ได้อีกวันละ ฿240" · "ชื่อของฉันนี้แล้วเหลือวันละ ฿180"	2 (รู้แค่เงินหายไปไหน)
C — AI ช่วยเข้าใจภาษาคน แต่ไม่แตะตัวเลข	AI อ่านประโยคและสลิป · การคำนวณเงินทุกบาทเป็นหน้าที่ของโค้ด	ความน่าเชื่อถือ

1.4 ทำไมเลือก LINE ไม่ใช่แอปของตัวเอง

เป็นการตัดสินใจเชิงออกแบบที่สำคัญที่สุดของโปรเจกต์ และต้องอธิบายให้ได้:

- นักศึกษาไทยเปิด LINE อยู่แล้วทั้งวัน — ต้นทุนการเปิดแอปเป็นศูนย์
- ไม่ต้องให้ผู้ผู้ใช้ติดตั้งอะไรเพิ่ม ไม่ต้องผ่าน App Store
- มี push notification ฟรี 280 ครั้ง/เดือน ไม่ต้องทำระบบแจ้งเตือนเอง
- ข้อแลกเปลี่ยนที่ยอมรับ: UI จำกัดอยู่ในกรอบของ LINE จึงต้องมีหน้าเว็บประกอบ

สำหรับงานที่ต้องเห็นภาพรวม (กราฟ ตาราง การแก้หลายรายการ)

1.5 ขอบเขต — อะไรอยู่ใน อะไรอยู่นอก

อยู่ใน (in scope)	อยู่นอก (out of scope) และเหตุผล
จดรายรับ-รายจ่ายส่วนบุคคล	การลงทุน/หุ้น — ต้องมีใบอนุญาตและความรับผิดชอบทางกฎหมาย
งบรายหมวด + เดือนก่อนใช้เกิน	เชื่อมบัญชีธนาคารโดยตรง (API ธนาคาร) — นักศึกษาขอ API ไม่ได้
แผนออม 3 ทางเลือก	หลายสกุลเงิน — ผู้ใช้เป้าหมายใช้บาทเดียว
อ่านอีเมลแจ้งเตือนธนาคาร	บัญชีกลุ่ม/ครอบครัว — เพิ่มความซับซ้อนเรื่องสิทธิมหาศาล
อ่านสลิปจากรูป	แซทเคย์เล่นทั่วไป — ไม่ใช่เป้าหมาย และเปลืองโควตา AI

2. แผนที่เอกสาร — ไฟล์ไหนคืออะไร

ก่อนลงโค้ด ต้องรู้ว่าเอกสารแต่ละฉบับตอบคำถามอะไร เพราะเวลาถูกถามแล้วตอบไม่ได้ จะหาคำตอบถูกที่

ไฟล์	ตอบคำถามว่า	ขนาด
docs/SRS.md	ต้องการอะไร — FR 20 ข้อ, NFR 11 ข้อ, เกณฑ์ยอมรับ	ฉบับไทย ละเอียดสุด
docs/M2-SRS_Lumen.pdf	เหมือนข้างบนแต่ย่อเหลือ 15 FR — ฉบับที่ส่ง M2 อาจารย์ก็อยู่	ภาษาอังกฤษ

ไฟล์	ตอบคำถามว่า	ขนาด
SPEC.md	จะสร้างอย่างไร — สถาปัตยกรรม S1-S13, สูตรคำนวณ, schema	ยาวสุดในโปรเจกต์
docs/RULES.md	กฎที่ห้ามฝ่าฝืน — กฎหลัก 7 ข้อ	1 หน้า
AID0.md	กติกากำกับ AI ตอนเขียนโค้ด	~750 บรรทัด
docs/FINAL_REPORT.md	รายงานฉบับสมบูรณ์ §1-§10 สำหรับส่ง M4	
docs/UI_CONTRACT.md	สัญญาระหว่าง API กับหน้าเว็บ — ฟิลต์ไหนหมายถึงอะไร	
docs/TASKS.md	ใครต้องทำอะไรต่อ	
docs/DEPLOY.md · EMAIL_SETUP.md	วิธี deploy และตั้งระบบอีเมล	
docs/M1-Charter_Lumen.pdf	Team Charter ส่ง M1	
docs/slides/Demo_Lumen.pdf	สไลด์นำเสนอ + SPEAKER_NOTES.md	
ไฟล์นี้	ระบบทำงานอย่างไรทั้งระบบ — สำหรับทีมอ่านเอง	

⚠ จุดที่ต้องรู้ไว้ก่อนถูกถาม: [docs/SRS.md](#) (ไทย) มี 20 FR ส่วน PDF ที่ส่ง M2 ย่อเหลือ 15 FR เลข FR จึงไม่ตรงกัน เช่น "อ่านสลิป" เป็น FR-15 ในฉบับไทย แต่เป็น FR-14 ใน PDF ถ้าถูกถามให้ยึดเลขตาม PDF เพราะนั่นคือฉบับที่กรรมการมีอยู่ในมือ

3. กฎหลัก 7 ข้อ — หัวใจของโปรเจกต์

นี่คือส่วนที่ต้องอธิบายให้ได้มากที่สุด เพราะเป็นสิ่งที่ทำให้โปรเจกต์นี้ต่างจาก "แอปจดเงินที่มี AI" ทั่วไป

แนวคิดตั้งต้น: AI เขียนโค้ดเร็วมาก แต่ ความเร็วไม่ใช่ปัญหาของแอปการเงิน ความถูกต้องคือปัญหา เราจึงเขียนกฎ 7 ข้อไว้ก่อนเขียนโค้ด แล้วบังคับใช้ผ่านโครงสร้าง ไม่ใช่ผ่านวินัยของคนเขียน

กฎ	ใจความ	บังคับใช้ที่ไหนในโค้ด (ชี้ได้จริง)
G1	AI ไม่คิดเลขเงิน	services/ai/tools.ts แปลงบาท→สตางค์ด้วย money.ts เท่านั้น · router.ts::mentionsUnseenNumber() ทั้งคำตอบที่มีตัวเลขซึ่งผู้ใช้ไม่ได้พิมพ์ · dedup.service.ts เขียนกำกับว่าห้ามมี AI
G2	AI แตะข้อมูลได้เมื่อคนกดยืนยัน	AI เขียนได้แค่แถวใน pending_actions · การเขียนจริงอยู่ใน pending.service.ts::confirmPending() ที่เรียกได้จาก postbackHandler (ผู้ใช้กดปุ่ม) เท่านั้น
G3	เงินเป็นจำนวนเต็ม สตางค์ บวกเสมอ ห้าม float	utils/money.ts แปลงด้วย string + BigInt ไม่ใช่ × 100 · assertValidTransactionAmount() · CHECK (amount > 0) ใน DB
G4	ปิด AI แล้วแอปต้องยังใช้ได้	ai/guard.ts ด้านแรกเช็ค AI_ENABLED · ทุกเส้นทางที่ AI ล้มคิน unavailable แล้ว textHandler ตอบคำแนะนำทางด่วน
G5	ส่งข้อมูลให้ AI เท่าที่จำเป็น	utils/redact.ts ปิดเลข 9-19 หลักก่อนส่ง · ประวัติแชทเก็บใน memory 3 turn / 30 นาที ไม่ลง DB · ai_usage_log ไม่มีคอลัมน์เก็บข้อความ · รูปสลิปไม่ถูกเขียนลงดิสก์
G6	ทุก query กรอง user_id	ทุกไฟล์ใน db/queries/ มี .eq('user_id', ...) · ทุกตารางเปิด RLS โดยไม่มี policy เลย

กฎ	ใจความ	บังคับใช้ที่ไหนในโค้ด (ชี้ได้จริง)
G7	<code>transfer</code> ไม่ใช่รายรับหรือรายจ่าย	ทุกสูตรใน <code>summary.service</code> / <code>plan.service</code> กรอง type ออก <code>CHECK (type = 'transfer' or plan_id is null)</code> ใน DB

3.1 ทำไมกฎแต่ละข้อถึงจำเป็น — คำตอบสำหรับ "ทำไมไม่ให้ AI คิดเลขไปเลย"

- G1 — ถ้าให้ AI คิดเลข เราจะไม่มีความรู้ว่ามันคิดผิด AI ตอบ "เดือนนี้ใช้ไป 3,200 บาท" ด้วยน้ำเสียงมั่นใจเท่ากันทั้งตอนถูกและตอนผิด ผู้ใช้ไม่มีทางแยกได้ และเราก็ไม่มีทางเขียนเทสต์ให้มันได้ เมื่อสูตรอยู่ในโค้ด เราเขียนเทสต์ได้ว่า "ใส่ข้อมูลชุดนี้ ต้องได้ตัวเลขนี้เป๊ะ"
- G2 — AI เข้าใจผิดได้ และเงินที่หายไปเอาคืนยาก `เมื่อซื้อกาแฟมาแก้วนี้ 80 บาท` อาจถูกอ่านเป็น 8 บาทหรือ 800 บาท การให้คนกดยืนยันคือการเอาคนกลับมาอยู่ในจุดตรงจุดที่สำคัญที่สุด
- G3 — `0.1 + 0.2 !== 0.3` ใน JavaScript เก็บเงินเป็นทศนิยมแล้วบวกกันหลายพันครั้ง ยอดจะเพี้ยนที่เศษสตางค์ เก็บเป็นสตางค์ (จำนวนเต็ม) แล้วแปลงตอนแสดงผลเท่านั้น
- G4 — AI มีโควตา มีวันล่ม และมีวันที่ Google ปิดรุ่น (เรื่องนี้เกิดขึ้นจริงในโปรเจกต์นี้ — ดู §13 คำถามที่ 9) ถ้าแอปพึ่ง AI แล้วเจตเงินไม่ได้ตอน AI ล่ม แอปนั้นใช้ไม่ได้จริง
- G5 — สลิปโอนเงินมีทั้งชื่อคน เลขบัญชี และยอดเงิน หลุดครั้งเดียวเอากลับไม่ได้
- G6 — เราเปิด RLS ทุกตารางแต่ไม่สร้าง policy เลย ผลคือ anon key เข้าไม่ได้แม้แต่แถวเดียว มีแค่ service_role key ที่ backend ใช้ ที่ข้าม RLS ได้ ดังนั้นการกรอง `user_id` ให้ถูกคนเป็นความรับผิดชอบของโค้ด 100% ไม่มีตาข่ายรองรับ ถ้าลืมกรองที่ไหนที่หนึ่ง ข้อมูลข้ามคนทันที
- G7 — เงินที่ย้ายเข้าแผนออมยังเป็นเงินของผู้ใช้ ไม่ได้ "ใช้ไป" ถ้านับเป็นรายจ่าย ยอด "ใช้ได้อีกวันละเท่าไร" จะต่ำกว่าจริง และผู้ใช้จะรู้สึกว่าการออมทำให้ตัวเองจนลง ซึ่งผิดทั้งในแง่ตัวเลขและในแง่พฤติกรรม

4. ภาพรวมสถาปัตยกรรม

4.1 ภาพใหญ่



4.2 เทคโนโลยีที่ใช้ และเหตุผลที่เลือก

ส่วน	ใช้อะไร	ทำไม
ภาษา	TypeScript (Node 22)	type ช่วยจับความผิดพลาดเรื่องหน่วยเงินตอน compile ไม่ใช่ตอน runtime
เว็บเซิร์ฟเวอร์	Express 4	เบา ทีมคุ้นเคย ไม่ต้องเรียน framework ใหม่ในเวลาจำกัด
ฐานข้อมูล	Supabase (Postgres)	ฟรี · มี RLS · เข้าถึงผ่าน HTTP ไม่ต้องจัดการ connection pool
บอท	@line/bot-sdk	SDK ทางการ มีตัวตรวจลายเซ็น webhook ให้
AI	Google Gemini ผ่าน @google/genai	free tier ไม่ต้องผูกบัตร · รองรับ function calling + อ่านรูป
เทสต์	Vitest	เร็ว ตั้งค่าน้อย รองรับ TS ตรง ๆ
โฮสต์	Render (free tier)	deploy จาก git ได้เลย
งานตามเวลา	GitHub Actions	Render free tier ไม่มี cron ให้

4.3 สี่ทางที่ข้อมูลเข้าระบบ

ข้อสังเกตสำคัญ: ทุกทางลงเอยที่ตาราง `transactions` ตารางเดียว — นี่คือเหตุผลที่ ยอดสรุปจากทุกช่องทางตรงกันเสมอ

#	ทางเข้า	ไฟล์ที่รับ	source ใน DB	parsed_by
1	พิมพ์ในแชท	handlers/textHandler.ts	chat	regex หรือ ai
2	อีเมลธนาคาร	services/email.service.ts	email	regex
3	รูปสลิป	handlers/imageHandler.ts	image	ai
4	หน้าเว็บ	routes/api.ts	liff	manual
5	รายการประจำ	services/recurring.service.ts	recurring	manual

4.4 Router 4 ชั้น — หัวใจของการ "จดง่าย"

เมื่อผู้ใช้พิมพ์ข้อความ ระบบไล่ 4 ชั้นจากถูกที่สุดไปแพงที่สุด ลำดับนี้สลับไม่ได้ และต้องอธิบายเหตุผลได้:

ข้อความเข้ามา

L0 คำสั่งตายตัว: สรุป · เหลือ · งบ · แผน · ยกเลิก · ออม <จำนวน> · ช่วยเหลือ · ความเป็นส่วนตัว
→ ถูกที่สุด แม่นที่สุด ไม่มีทางผิด
⚠ ต้องมาก่อน L1 ไม่งั้น "สรุป" จะถูกอ่านเป็นชื่อรายการที่ไม่มียอด

L1 ทางด่วน regex: "กาแฟ 80" · "กาแฟ ห้าสิบ" · "ค่านีต 1.2k" · "+เงินเดือน 35000"
→ ฟรี ไม่ต้องรอ network ครอบคลุมข้อความส่วนใหญ่
ไฟล์: utils/regexParser.ts + utils/thaiNumber.ts

L2/L3 พจนานุกรมหมวด (กลาง + ส่วนตัว)
→ ⚠ ยังไม่ได้เขียน (services/keyword.service.ts เป็นไฟล์ว่าง)
ตอนนี้ใช้การจับคู่กับหมวดที่ผู้ใช้มีอยู่แล้วแทน

L4 AI (Gemini): ประโยคยาว คำถาม คำสั่งแก้/ลบ
→ แพงที่สุด มีโควตา ต้องผ่าน guard 5 ด่าน
ไฟล์: services/ai/router.ts

ทำไมต้องมี 4 ชั้น ไม่ส่งทุกอย่างให้ AI เลย:

- 1. ค่าใช้จ่ายและโควตา — free tier จำกัดจำนวนครั้งต่อวัน ถ้าส่งทุกข้อความไปจะหมดเร็ว
- 2. ความเร็ว — regex ตอบทันที AI ใช้เวลาหลายวินาที
- 3. ความแม่นยำ — `ภาพ 80` regex อ่านถูก 100% ส่วน AI อ่านถูก "เกือบ" 100%
- 4. G4 — ถ้า AI ล่ม L0 กับ L1 ยังทำงาน = แอปยังใช้ได้

5. ไลค์ดทีละชั้น

5.1 หลักการจัดชั้น (layering) — กฎเดียวที่ต้องจำ

routes/	← รับ HTTP, ตรวจสอบ input, แปลง error เป็น status code (ไม่มี business logic)
handlers/	← รับ event จาก LINE, ตัดสินว่าจะตอบอะไร (ไม่คำนวณเงิน)
services/	← business logic + การคำนวณทั้งหมด (ไม่รู้จักร HTTP/LINE)
db/queries/	← คุยกับ Supabase เท่านั้น (ไม่มี logic)
utils/	← ฟังก์ชันบริสุทธิ์ ไม่แตะ I/O (ทดสอบง่ายสุด)

กฎ: ชั้นบนเรียกชั้นล่างได้ ชั้นล่างห้ามเรียกชั้นบน ประโยชน์ที่ได้จริง: เขียนเทสต์ service ได้โดย mock แด่ `db/queries/` ไม่ต้องต่อ DB จริง ซึ่งเป็นเหตุผลที่เทสต์ 683 ข้อรันจบใน ~7 วินาที

5.2 `src/config/` — ค่าคงที่และการตั้งค่า

ไฟล์	ทำอะไร	สิ่งที่ต้องรู้
<code>env.ts</code>	โหลด env แล้ว validate ด้วย Zod	ถ้า env ไม่ครบจะพังตั้งแต่ boot พร้อมบอกข้อผิดพลาด — เจตนาคือไม่ให้ค่าที่หายไป "เงียบ" แล้วไปโผล่เป็น error แปลก ๆ ที่หลัง
<code>constants.ts</code>	ค่าคงที่ของธุรกิจ	ดูตารางใน §14 — ถ้าแก้ค่าพวกนี้ต้องแก้ <code>SPEC.md</code> คู่กันเสมอ
<code>privacy.ts</code>	ข้อความแจ้ง PDPA + ข้อความต้อนรับ	อยู่ที่นี่เพื่อไม่ให้ข้อความในแชทกับในคำสั่งไม่ตรงกัน
<code>keywords.ts</code>	พจนานุกรมหมวดกลาง (L2)	ยังเป็นโครงเปล่า

5.3 `src/utils/` — ฟังก์ชันบริสุทธิ์ (ชั้นล่างสุด)

ชั้นนี้ไม่แตะ DB ไม่แตะ network เลย จึงเทสต์ได้ตรง ๆ และเป็นทีที่เทสต์หนาที่สุด

ไฟล์	ทำอะไร	ทำไมต้องมี · จุดที่ต้องรู้
<code>money.ts</code> (216 บรรทัด)	แปลงบาท↔สตางค์ · บวก · หัก · บิล · ปิดเศษ	ไฟล์สำคัญที่สุดในโปรเจกต์ · <code>toSatang()</code> ใช้ string-parsing + BigInt ไม่ใช่ <code>× 100</code> เพราะ float พี้ยน · <code>splitEvenly(10000, 3) = [3334, 3333, 3333]</code> รวมได้ 10000 เป๊ะ ไม่ใช่หารแล้วเศษหาย
<code>thaiNumber.ts</code> (204)	อ่านจำนวนเงินภาษาไทย	<code>"ห้าสิบ"</code> → 5000 สตางค์ · <code>"1.2k"</code> → 120000 · <code>"สองพันห้า"</code> → 250000
<code>thaiDate.ts</code> (196)	คำนวณวันที่ตามเวลาไทย	คำนวณด้วย <code>Date.UTC</code> ล้วน แล้วอ่านกลับด้วย <code>getUTC*</code> → ผลเหมือนกันทุกเครื่องไม่ว่า TZ ของเซิร์ฟเวอร์เป็นอะไร

ไฟล์	ทำอะไร	ทำไมต้องมี · จุดที่ต้องรู้
<code>thaiDateText.ts</code> (241)	แปลงคำบอกวันเป็นวันที่จริง	"เมื่อวาน" · "1 ต.ค." · "3 วันก่อน" → ISO date · อ่านไม่ออกคืน <code>null</code> ไม่ใช่เดาเป็นวันนี่
<code>regexParser.ts</code> (139)	ทางด่วน L1	คืน <code>confidence</code> มาด้วย — <code>high</code> เท่านั้นที่บันทึกเลย
<code>redact.ts</code> (26)	ปิดเลขบัญชี/เบอร์โทรก่อนส่ง AI	จับเลข 9-19 หลัก ที่ค้นด้วยวันวรรค/ขีดได้ · จงใจไม่นับ comma เพราะ comma ใช้ค้นหลักพันของเงิน ซึ่งต้องส่งให้ AI อ่าน
<code>session.ts</code> (115)	เซ็น/ตรวจ session cookie	เขียนเอง ไม่ใช่ <code>express-session</code> เพราะไม่ต้องการ session store · ข้อแลกเปลี่ยน: แต่ละคนออกทีละคนไม่ได้ ต้องเปลี่ยน <code>SESSION_SECRET</code> ซึ่งแต่ละทุกคนพร้อมกัน
<code>logger.ts</code> (12)	log แบบมีรูปแบบ	

5.4 `src/db/queries/` — ชั้นคุย Supabase

กฎของชั้นนี้: ไม่มี business logic เลย แปลงพารามิเตอร์เป็น query แล้วคืนแถวดิบ ทุกฟังก์ชันมี `.eq('user_id', ...)` ตาม G6

ไฟล์	ตารางที่ดูแล	จุดที่ต้องรู้
<code>transactions.ts</code> (363)	<code>transactions</code>	ใหญ่สุด · มี query สำหรับ dedup, soft delete, restore
<code>plans.ts</code> (224)	<code>plans</code>	
<code>categories.ts</code> (223)	<code>categories</code>	<code>findOrCreateCategory()</code> สร้างหมวดให้ถ้ายังไม่มี — จุดที่ AI ห้ามเรียกตรง ๆ (ดู §7.4)
<code>recurring.ts</code> (195)	<code>recurring_rules</code>	
<code>budgets.ts</code> (161)	<code>budgets</code>	เก็บธวงว่าเดือน 80%/100% ไปแล้วหรือยัง กันเดือนซ้ำ
<code>users.ts</code> (161)	<code>users</code>	รวม <code>ai_enabled</code> (ทางปิด AI ของผู้ใช้) และ email token
<code>pending.ts</code> (156)	<code>pending_actions</code>	<code>claimPendingAction()</code> จองแถวแบบ atomic — หัวใจของการกันกดซ้ำ
<code>summary.ts</code> (124)	อ่านข้ามตาราง	
<code>logs.ts</code> (87)	<code>ai_usage_log</code>	ไม่มีคอลัมน์เก็บข้อความผู้ใช้ และจะไม่เพิ่ม ตาม G5
<code>emails.ts</code> (82) · <code>push.ts</code> (81) · <code>cleanup.ts</code> (74) · <code>keywords.ts</code> (7)	ตามชื่อ	

5.5 `src/services/` — business logic ทั้งหมด

นี่คือชั้นที่คำนวณเงิน และเป็นชั้นที่เทสต์เล็งเป็นหลัก

ไฟล์	ทำอะไร	ทำไมต้องมี
<code>plan.service.ts</code> (663)	แผนออก: capacity · 3 ทางเลือก · โอนเข้าแผน · ความคืบหน้า	ตอบคำถามที่ 2 ของโปรเจกต์

ไฟล์	ทำอะไร	ทำไมต้องมี
<code>pending.service.ts</code> (474)	วงจรคำขอรอยืนยัน waiting→confirmed/cancelled	ที่เดียวที่คำขอของ AI กลายเป็นข้อมูลจริง (G2)
<code>recurring.service.ts</code> (412)	รายการประจำ + รอบถัดไป	<code>shiftMonthClampDay</code> จัดการกรณี 31 ม.ค. → 28 ก.พ. → 31 มี.ค. (ไม่ใช่ 28)
<code>budget.service.ts</code> (298)	งบรายหมวด + เดือน 80%/100%	เดือนระดับละครั้งเดียวต่อเดือนด้วยธงใน DB
<code>command.service.ts</code> (287)	คำสั่งตายตัวทั้งหมด (L0)	จับแบบ ตรงทั้งข้อความ ไม่ใช่ <code>includes()</code> ไม่จิ้น "จ่ายค่าสรุปโครงการ 200" จะถูกมองว่าเป็นคำสั่ง
<code>email.service.ts</code> (268)	ดึงอีเมล → แยกยอด → dedup → บันทึกลง	
<code>transaction.service.ts</code> (245)	สร้าง/แก้/ลบรายการ	ประเด็นเดียวที่เขียน <code>transactions</code> — ไม่มีใครเขียนตารางนี้ข้ามไฟล์นี้ได้
<code>summary.service.ts</code> (217)	ยอดสรุป + safe-to-spend + กราฟ 6 เดือน	ตอบคำถามที่ 1 ของโปรเจกต์
<code>category.service.ts</code> (193)	จัดการหมวด	<code>updateCategory</code> ปฏิเสธการเปลี่ยน <code>type</code> เพราะจะพลิกความหมายของทุกรายการที่ผูกอยู่ย้อนหลัง
<code>planCheck.service.ts</code> (163)	เช็ครายวันว่าแผนยังไหวไหม	
<code>simulate.service.ts</code> (152)	จำลอง "ซื้อแล้วงบจะเหลือเท่าไร"	นับวันแบบเดียวกับ safe-to-spend เป๊ะ เพื่อให้สองหน้าไม่ขัดกัน
<code>quota.service.ts</code> (79)	คุมโควตา push 280/เดือน	
<code>dedup.service.ts</code> (66)	ตัดสินใจว่ารายการซ้ำไหม	🔴 ห้ามมี AI · มีแค่ 2 เงื่อนไข (ดู §5.6)
<code>cleanup.service.ts</code> (62)	ลบของเก่า	๑ ห้ามแตะ <code>transactions</code> — เงินของผู้ใช้ไม่มีวันหมดอายุ
<code>keyword.service.ts</code> (7)	พจนานุกรม L3	☐ ยังไม่ได้เขียน
<code>ai/</code> 6 ไฟล์	ชั้น AI ทั้งหมด	ดู §7
<code>email/</code> 5 ไฟล์	DKIM · token ผู้รับ · parser ของ KBank	

5.6 S10 Dedup — มีแค่ 2 เงื่อนไข และห้ามเพิ่ม

เงื่อนไข 1: <code>ref_number</code> เท่ากัน	→ ซ้ำแน่นอน (exact)
เงื่อนไข 2: ยอดเท่ากันเป๊ะ + <code>type</code> เดียวกัน + เวลาห่างกันไม่เกิน 30 นาที	→ น่าจะซ้ำ (probable)

`SPEC.md` ปิดท้ายหัวข้อนี้ว่า "ห้ามรวมรายการอัตโนมัติด้วยเงื่อนไขอื่นนอกจากนี้ การรวมผิดทำให้ข้อมูลหายแบบไม่มีร่องรอย"

ต้องอธิบายให้ได้ว่าทำไมไม่ใส่เงื่อนไข "ฉลาด ๆ" เช่นเทียบชื่อร้านค้าคล้ายกัน: การตัดสินใจว่า "ซ้ำ" ทำให้รายการจริงหายไปโดยไม่มี log ไม่มีใครรู้ ต่างจากการตัดสินใจว่า "ไม่ซ้ำ" ที่ผู้ใช้เห็นรายการคู่แล้วลบเองได้ ความเสียหายไม่สมมาตร จึงเลือกพลาดไปทางที่กู้คืนได้

5.7 `src/handlers/` — รับ event จาก LINE

ไฟล์	รับ event	ทำอะไร
<code>textHandler.ts</code> (213)	ข้อความ	ไล่ L0 → L1 → L4 ตาม §4.4
<code>postbackHandler.ts</code> (228)	กดปุ่ม	<code>undo</code> · <code>restore</code> · <code>save_to_plan</code> · <code>ai_confirm</code> / <code>ai_cancel</code> ← จุดที่ G2 เป็นรูปธรรม
<code>imageHandler.ts</code> (211)	รูป	วงจรอ่านสลิปทั้งเส้น (ดู §7.5)
<code>followHandler.ts</code> (126)	แอดเพื่อน/ บล็อก	สร้างบัญชี + หมวดเริ่มต้น 11 หมวด + ส่งข้อความแจ้ง PDPA

5.8 `src/routes/` — ชั้น HTTP

ไฟล์	เส้นทาง	จุดที่ต้องรู้
<code>api.ts</code> (619)	<code>/api/*</code> 28 endpoint	ทุกเส้นผ่าน <code>requireSession</code> ก่อน · มีฟิลด์ <code>unavailable</code> บอกตรง ๆ ว่าจะไวยังคำนวณไม่ได้ แทนที่จะส่ง 0 มาหลอก
<code>webhook.ts</code> (122)	<code>POST /webhook</code>	ตอบ 200 ทันทีก่อนประมวลผล (LINE ต้องการ < 1 วิ) · กั้น event ซ้ำด้วยตาราง <code>webhook_events</code>
<code>auth.ts</code> (191)	<code>/auth/login</code> <code>/callback</code> <code>/logout</code>	LINE Login OAuth
<code>jobs.ts</code> (77)	<code>POST /jobs/run?job=</code>	ต้องมี header <code>x-cron-secret</code>
<code>health.ts</code> (47)	<code>GET /health</code>	บอกว่า env ครบไหม — ตัวช่วย debug ที่ใช้บ่อยที่สุดตอน deploy

5.9 ทางเดินของข้อความหนึ่งข้อความ (ตัวอย่างจริง)

ผู้ใช้พิมพ์ `กาแฟ 80` — ไล่ตามโค้ดได้ดังนี้ (ใช้ช้อมูลอธิบายได้เลย):

1. LINE อิง POST `/webhook` มาพร้อม header `x-line-signature`
2. `index.ts` → middleware ของ SDK ตรวจสอบเซ็นด้วย `LINE_CHANNEL_SECRET`
ไม่ผ่าน = 401 จบ (ป้องกันคนอื่นปลอม event)
3. `webhook.ts` → ตอบ 200 ทันที แล้วค่อยประมวลผลข้างหลัง
insert `webhookEventId` ลง `webhook_events` ชวนซ้ำ = ซ้ำม (idempotent)
4. `textHandler` → `getUserByIdByLineUserId()` แปลง LINE id เป็น `user_id` ของเรา (G6)
5. → `handleCommands()` เช็ค L0 ก่อน — "กาแฟ 80" ไม่ใช่คำสั่ง ผ่าน
6. → `parseQuickExpenseText("กาแฟ 80")`
regexParser + thaiNumber → { category:"กาแฟ", amountSatang: 8000, type:"expense", confidence:"high" }
7. → `createTransaction()` ใน `transaction.service`
 - └ `assertValidTransactionAmount(8000)` ตรวจ G3
 - └ `findOrCreateCategory()` หาหมวด
 - └ `insertTransaction()` เขียน DB
 - └ ประเมินว่าข้ามเกณฑ์ทั้ง 80%/100% ไหม
8. → `buildConfirmCard()` สร้าง Flex message
9. → `replyFlex()` ตอบกลับ พร้อมแนบค่าเตือนงบถ้ามี

จุดที่ควรชี้ให้กรรมการเห็น: ค่าเตือนงบถูกแนบไปกับ reply เดียวกัน ไม่ใช่ส่ง push แยก เพราะ reply ฟรีไม่จำกัด แต่ push มีแค่ 280 ครั้ง/เดือนทั้งระบบ

6. Money Engine (S5) — เขตห้าม AI

นี่คือส่วนที่ต้องอธิบายได้ละเอียดที่สุด เพราะเป็นแกนของคำถามทั้งสองข้อ ไฟล์ในส่วนนี้ไม่ import อะไรเกี่ยวกับ AI เลยแม้แต่ตัวเดียว — นั่นคือวิธีบังคับ G1 ผ่านโครงสร้าง ไม่ใช่ผ่านวินัย

6.1 Safe-to-spend — "ใช้ได้อีกวันละเท่าไร" (ตอบคำถามที่ 1)

ไฟล์: `services/summary.service.ts::getSafeToSpend()`

```
เงินเหลือเดือนนี้ = รายรับที่เข้าแล้ว
                  + รายรับประจำที่ยังจะเข้าในเดือนนี้
                  - รายจ่ายที่ใช้ไปแล้ว
                  - รายจ่ายประจำที่ยังจะออกในเดือนนี้
                  - ยอดที่ผูกไว้กับแผนออมที่ active

ใช้ได้วันละ = ปัดลง( เงินเหลือเดือนนี้ ÷ จำนวนวันที่เหลือรวมวันนี้ )
```

ตัวอย่างตัวเลข (ใช้ตอบได้เลยถ้าถูกถาม): วันที่ 17 ของเดือนที่มี 30 วัน → เหลือ 14 วัน (`30 - 17 + 1`) ถ้าเงินเหลือ 3,360 บาท → ใช้ได้วันละ 240 บาท

4 จุดที่ต้องอธิบายได้ว่าทำไมทำแบบนี้:

1. ทำไมหักรายการประจำที่ยังไม่เกิด — ค่าหอทที่จะออกวันที่ 25 เป็นเงินที่ "มีเจ้าของแล้ว"

ถ้าไม่หักออก ผู้ใช้จะใช้จ่ายเงินก่อนหน้านี้ไปแล้วค่อยมารู้ตัวตอนสิ้นเดือน

1. ทำไมหักยอดแผนออม — เงินที่สัญญากับตัวเองว่าจะเก็บ ไม่ควรถูกนับว่าใช้ได้

ไม่เงินแผนออมจะพึงเจียบ ๆ ทุกเดือน

1. ทำไม "ปัดลง" ไม่ใช่ปัดครึ่งขึ้น — ปัดขึ้นแล้วผู้ใช้จะใช้จ่ายเกินที่ละเศษสตางค์ทุกวัน

สะสม 30 วันแล้วเกินงบจริง การปัดลงผิดไปในทางที่ปลอดภัย

1. ทำไมนับวันนี้ด้วย — วันนี้ยังใช้เงินได้ ถ้าไม่นับ ตัวเลขจะสูงเกินจริง

6.2 Plan Capacity — "เก็บได้เดือนละเท่าไร" (ตอบคำถามที่ 2)

ไฟล์: `services/plan.service.ts::getPlanCapacity()`

```
ดูข้อมูลย้อนหลัง 90 วัน (LOOKBACK_DAYS)

เงินเหลือใช้ (disposable) = รายรับเฉลี่ย/เดือน
                          - รายการประจำ/เดือน
                          - รายจ่ายจำเป็นเฉลี่ย/เดือน (หมวดที่ is_essential)
                          - เงินกันฉุกเฉิน (15% ของรายจ่ายเฉลี่ย)

ออมได้จริง (capacity) = ปัดครึ่งขึ้น( disposable × 0.8 ) - ยอดที่ผูกกับแผนเดิมอยู่แล้ว
```

ทำไมคูณ 0.8 (`PLAN_SAFETY_RATIO`) — ค่าเฉลี่ยย้อนหลังไม่ใช่ค่าทำนาย เดือนหน้าอาจมี รายจ่ายที่ไม่เคยเกิด การกินพื้นที่ไว้ 20% ทำให้แผนที่เสนอไป "โหวตจริง" มากกว่าแผนที่สวยบนกระดาษ

ทำไมกันฉุกเฉิน 15% (`EMERGENCY_BUFFER`) — แผนออมที่ทำให้ไม่มีเงินสำรองเลย เป็นแผนที่พังเมื่อเจอค่าใช้จ่ายฉุกเฉินครั้งเดียว

6.3 สามทางเลือก — เร็ว / สมดุล / สบาย

ทางเลือก	ยอดออม/เดือน	เหมาะกับ
< เร็ว	ตามเวลาที่ผู้ใช้ระบุ (ถ้าไหว) หรือเท่า capacity เต็ม	คนที่รีบ
สมดุล	capacity × 0.6 (<code>BALANCED_RATIO</code>)	ค่ากลาง
 สบาย	capacity × 0.35 (<code>RELAXED_RATIO</code>)	คนที่กลัวพัง

ทำไมต้องให้ 3 ทางเลือกไม่ใช่คำตอบเดียว — การเงินส่วนบุคคลไม่มีคำตอบถูกข้อเดียว คนที่รับความกดดันได้ต่างกัน การเสนอทางเลือกพร้อมผลที่ตามมา (ใช้เวลากี่เดือน) ให้ผู้ใช้เป็นคนตัดสินใจ ซึ่งตรงกับเจตนาของ G2 ในระดับการออกแบบ

ระดับความมั่นใจ (confidence) ตาม S5.5 — cold start:

ข้อมูลย้อนหลัง	confidence	ระบบทำอะไร
< 7 วัน	<code>low</code>	ถ้าไม่มีรายรับประจำ ยังไม่สร้างแผน ขอให้ตั้งรายการประจำก่อน
7-90 วัน	<code>medium</code>	สร้างแผนได้ แต่บอกว่าข้อมูลยังน้อย
> 90 วัน	<code>high</code>	เชื่อค่าเฉลี่ยได้

ทำไมต้องมี cold start — ผู้ใช้ใหม่ที่เพิ่งจด 2 รายการ ถ้าเอาค่าเฉลี่ยมาคำนวณจะได้ ตัวเลขที่ไร้ความหมาย การบอกว่า "ยังไม่รู้รายได้ประจำของคุณ" ตรงไปตรงมาดีกว่า เสนอแผนที่คำนวณจากข้อมูล 2 แถว

6.4 เหตุผลที่ระบบปฏิเสธการสร้างแผน — ต้องเป็นตัวเลขจับต้องได้

โค้ดส่วนนี้ (`plan.service.ts` ราวบรรทัด 250) จงใจไม่ตอบ "ข้อมูลไม่พอ" ลอย ๆ แต่ตอบว่า:

- ยังไม่รู้รายได้ประจำของคุณ — ตั้งรายการประจำของรายรับก่อน
- มีแผนที่กำลังออมอยู่ 3 แผนแล้ว (สูงสุด 3) — ปิดแผนเดิมก่อน
- เงินที่ออมได้ถูกใช้กับแผนเดิมหมดแล้ว — ลดยอดออมของแผนเดิมหรือยืดเวลาออกไป
- รายรับหลังหักรายจ่ายจำเป็นยังไม่เหลือพอให้ออม — ลองลดรายจ่ายที่ไม่จำเป็นลงก่อน

เหตุผลเชิงออกแบบ: ข้อความที่บอกทางไปต่อได้ทำให้ผู้ใช้แก้ปัญหาเองได้ ข้อความที่บอกแค่ว่าผิดพลาดทำให้ผู้ใช้เลิกใช้

6.5 งบรายหมวด + เดือน 80%/100%

ไฟล์: `services/budget.service.ts`

เดือนตอน 80% (ยังแก้ทัน) และ 100% (เกินแล้ว) โดยเดือน ระดับละครึ่งเดียวต่อเดือน ด้วยธงที่เก็บใน DB

จุดที่ต้องรู้: การเตือนถูกแนบไปกับ reply ของรายการที่ทำให้ข้ามเกณฑ์พอดี ไม่ได้ส่งเป็น push แยก — เพราะ reply ฟรี ส่วน push มีโควตา 280/เดือน

7. ชั้น AI — ให้ AI ช่วยได้ แต่ไม่ให้แตะเงิน

7.1 โครงสร้าง 6 ไฟล์

```
services/ai/
├─ guard.ts      (122)  ด้านตรวจสอบเรียก AI ทุกครั้ง — 5 ชั้นตามลำดับ
├─ prompt.ts     (56)   system prompt + ข้อความ fallback ตอน AI ใช้ไม่ได้
├─ tools.ts      (614)  ประกาศ 8 tools ให้ Gemini + Zod + sanity check
├─ gemini.ts     (211)  T1 understandText — อ่านข้อความ (function calling)
├─ vision.ts     (433)  T2 readSlip — อ่านสลิปจากรูป
└─ router.ts     (501)  ตัวประกอบร่าง: guard → AI → ตรวจ → pending/ตอบ
```

มีแค่ `gemini.ts` กับ `vision.ts` ที่เรียก Gemini ได้ (และ `tools.ts` ที่ใช้แค่ type) และมี เทสต์เชิงโครงสร้าง ใน `tests/aiRouter.test.ts` ที่อ่านไฟล์ทั้ง `src/` แล้วยืนยันว่า ไม่มีไฟล์อื่น import `@google/genai` — ถ้าวันหลังมีคนเพิ่ม เทสต์จะล้มทันที

7.2 Guard 5 ด้าน — ลำดับสลับไม่ได้

```
1. AI_ENABLED = false ?           → ปฏิเสธทันที ไม่แตะ DB เลย           (G4)
2. ไม่มี GEMINI_API_KEY ?         → ปฏิเสธ ไม่แตะ DB
3. users.ai_enabled = false ?     → ผู้ใช้ปิดเอง                           (PDPA)
4. เกินโควตาต่อคนต่อวัน (30) ?   → ปฏิเสธ
5. เกินโควตาทั้งระบบต่อวัน ?     → ปฏิเสธ

ผ่านแล้ว → redact ข้อความ แล้วคืนเฉพาะข้อมูลที่ปิดข้อมูลแล้ว           (G5)
```

ทำไมลำดับนี้ — ด้าน 1-2 ไม่ต้อง query DB เลย ถ้าเอาด้านโควตาขึ้นก่อน ระบบที่ปิด AI ไว้ จะยัง query ไป DB ทุกข้อความที่ ผู้ใช้พิมพ์ ซึ่งเปลืองเปล่า ๆ

จุดสำคัญ: `guard` ห้าม throw เด็ดขาด ไม่ว่า DB จะล่มหรืออะไรจะเกิด ตอบ `{ ok: false }` เสมอ เพื่อให้ผู้เรียกไปใช้ทาง ที่ไม่ต้องมี AI ต่อได้ (G4) และถ้านับโควตาไม่ได้ ระบบเลือก "ปฏิเสธเมื่อไม่รู้" เพราะถ้าปล่อยผ่านตอน DB ล่ม โควตาของ provider อาจหมดในไม่กี่นาทีโดยไม่มีอะไรหยุดได้

7.3 8 Tools ที่ AI เลือกได้

#	tool	ชนิด	ผลที่เกิด
1	<code>create_transaction</code>	เขียน	→ pending รอกดยืนยัน
2	<code>create_transaction_batch</code>	เขียน	→ pending (สูงสุด 10 รายการ)
3	<code>update_transaction</code>	เขียน	→ pending แสดง "ก่อน → หลัง"
4	<code>delete_transaction</code>	เขียน	→ pending
5	<code>create_recurring</code>	เขียน	→ pending
6	<code>get_summary</code>	อ่าน	→ เรียก <code>runCommand()</code> ตัวเดียวกับคำสั่ง <code>สรุป</code>
7	<code>query_transactions</code>	อ่าน	→ รายการที่ค้นเจอ
8	<code>simulate_purchase</code>	อ่าน	→ ผลจำลองจาก <code>simulate.service</code>

จุดที่ควรชี้: tool #6 เรียกฟังก์ชันตัวเดียวกับที่คำสั่ง `สรุป` ใช้ ผลคือผู้ใช้ถามด้วยภาษาคนแล้วได้ตัวเลขชุดเดียวกับที่พิมพ์คำสั่ง เป๊ะ ๆ ไม่มี "สองความจริง" ในระบบ

7.4 สามด้านที่บังคับ G1 — AI ไม่มีทางแตะตัวเลขเงิน

ด้าน 1 — prompt (ขอ) บอกโมเดลว่า "ดึงเฉพาะตัวเลขที่ผู้ใช้พิมพ์ ห้ามคำนวณ ถ้าต้องหารให้ใส่ `split_count`" — แต่เนี่ยเป็นแค่การขอ โมเดลอาจไม่ทำตาม

ด้าน 2 — Zod + sanity check (บังคับ) `tools.ts` ตรวจทุก arg:

- `0 < amount ≤ 10,000,000 บาท` · แปลงด้วย `money.toSatang()` เท่านั้น
- `split_count` 2-50 · หารด้วย `splitEvenly()` ที่เศษสตางค์ไม่หาย
- `date_text` แปลงด้วย `thaiDateText` · อ่านไม่ออก = ถ้ามกลับ ไม่เดา

ด้าน 3 — ตรวจคำตอบแบบข้อความ `router.ts::mentionsUnseenNumber()` ถ้า AI ตอบเป็นข้อความที่มีตัวเลข 3 หลักขึ้นไปซึ่งผู้ใช้ไม่ได้พิมพ์มา → ทั้งคำตอบทั้งก่อน เพราะนั่นคือตัวเลขที่โมเดลแต่งขึ้นเอง เช่น "เดือนนี้คุณใช้ไป 3,200 บาท"

อีกจุดที่ป้องกันไว้ — ห้าม AI สร้างหมวดใหม่ `router.ts::resolveCategoryName()` จับคู่ `category_hint` กับหมวดที่ผู้ใช้มีอยู่แล้ว เท่านั้น ถ้าส่ง hint เข้า `findOrCreateCategory()` ตรง ๆ คำว่า "กาแฟ" "ค่ากาแฟ" "เครื่องดื่ม" จาก 3 ข้อความจะกลายเป็น 3 หมวดใหม่ แล้วหน้าสรุปรายหมวดจะพังในไม่กี่วัน หาไม่เจอ → รายจ่ายใช้ "อื่นๆ" · รายรับปล่อยว่าง (ไม่มีหมวด "อื่นๆ" ผังรายรับ และการโยนเงินเดือนเข้า "รายได้เสริม" คือการติดป้ายผิด)

7.5 วงจรอ่านสลิป (FR-14 ใน PDF / FR-15 ในฉบับไทย)

ผู้ใช้ส่งรูป

↓

1. หา user_id จาก line_user_id (G6)
2. โหลด loading animation (Vision ใช้หลายวินาที)
3. ดึงรูปจาก LINE Content API → เข้า Buffer ใน memory (G5 ห้ามเขียนลงดิสก์)
4. guard (kind='vision') → ไม่ผ่าน: บอกให้พิมพ์เอง (G4)
5. Gemini Vision → JSON (บังคับรูปด้วย responseSchema)
6. Zod + sanity: ยอด > 0 · วันที่ไม่อยู่ในอนาคต
7. ปิดเลขบัญชีในชื่อผู้รับ (G5)
8. เช็คซ้ำตาม S10 — ref_number ตรง = ไม่สร้างค่าขอเลย (ก่อนสร้าง pending)
9. สร้าง pending (source='image') (G2)
10. ตอบพร้อมปุ่ม ยืนยัน / ไม่ใช่

4 จุดที่ต้องอธิบายได้:

1. ทำไมไม่เก็บรูป — `SPEC.md` §S9 เขียนว่า "MUST NOT เก็บไฟล์รูปไว้ที่ใดเลย"

สลิปมีชื่อคน เลขบัญชี ยอดเงิน · มีเทสต์เชิงโครงสร้างอ่าน source ของ `imageHandler.ts` แล้วยืนยันว่าไม่มีคำว่า `writeFile` `createWriteStream` `node:fs` `/tmp`

1. ทำไมปิด `receiver` แต่ไม่ปิด `ref_number` — `redact` ปิดเลข 9 หลักขึ้นไป

ซึ่งคลุมเลขอ้างอิงพอดี ถ้าเผลอปิดช่องนั้น ฎยกั้นซ้ำที่แม่นยำสุดของ S10 จะใช้ไม่ได้

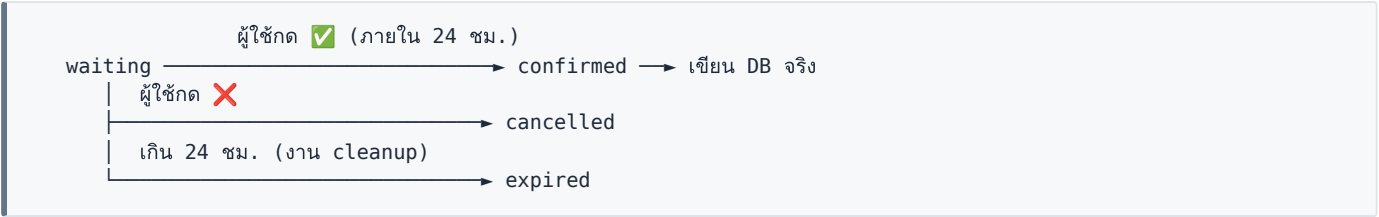
1. ทำไมเวลาบนสลิปตีความเป็นเวลาไทย — `14:32` บนสลิป → `07:32Z`

ถ้าตีความเป็น UTC รายการเลื่อน 7 ชั่วโมง พอทำให้รายการตอนเช้าตรู่ข้ามไปเป็นวันก่อนหน้า

1. ทำไมสลิปลงวันอนาคตถูกปฏิเสธ — เป็นอาการของการอ่านผิด (เช่นอ่านปี พ.ศ. 2569

เป็น ค.ศ. 2569) ไม่ใช่สลิปจริง

7.6 Pending Actions — ที่เดียวที่ AI กลายเป็นข้อมูลจริง



กลไกสำคัญที่สุด — การ "จอง" แลวแบบ atomic `db/queries/pending.ts::claimPendingAction()`

```
update pending_actions set status='confirmed'
where id=? and user_id=? and status='waiting' and expires_at > now()
returning *
```

ปัญหาที่มันแก้: ผู้ใช้กดปุ่ม ☒ สองครั้งรวดเร็ว (เน็ตช้าแล้วกดซ้ำ หรือ LINE ส่ง postback ซ้ำ) ถ้าเขียนแบบ "อ่านก่อนว่า waiting ไหม แล้วค่อยเขียน" จังหวะระหว่างอ่านกับเขียนจะมีช่องให้ คำขอที่สองอ่านเจอ `waiting` เหมือนกัน → บันทึกรายการซ้ำ 2 ครั้ง การใส่เงื่อนไขลงใน `UPDATE` เลยทำให้ฐานข้อมูลรับประกันว่าสำเร็จได้แค่คำขอเดียว

อีกจุดที่สำคัญ: ถ้าเขียน DB จริงล้มเหลว หลัง จองแลวแล้ว → คินสถานะเป็น `waiting` ไม่จันผู้ใช้กดยืนยันอีกครั้งไม่ได้เลยทั้งที่รายการยังไม่ได้ถูกบันทึก = เงินหายไปจากระบบโดยที่ผู้ใช้คิดว่าจดแลว

7.7 PDPA — การแจ้งผู้ใช้

`SPEC.md` §S11.4 สั่งไว้ว่าต้องอ่านเงื่อนไขของ Gemini API เรื่องการนำข้อมูลไปใช้ ปรับปรุงบริการ (โดยเฉพาะ free tier) แลว เขียนแจ้งผู้ใช้ในข้อความต้อนรับให้ตรงกับความจริง

เราทำครบ 3 อย่าง:

- 1. ข้อความต้อนรับ ตอนแอดเพื่อน บอกว่ามี AI · ปิดเลขบัญชีก่อนส่ง · ปิดได้
- 2. คำสั่ง `ความเป็นส่วนตัว` อ่านประกาศฉบับเต็มซ้ำได้ตลอด (ข้อความต้อนรับเลื่อนหายในแชท)
- 3. สวิตช์ปิด AI จริง หน้าเว็บ → ตั้งค่า → เขียน `users.ai_enabled` ซึ่ง guard ด่าน 3 อ่าน

ประกาศบอกตรง ๆ ว่า free tier ของ Google อาจนำข้อมูลไปใช้ปรับปรุงบริการได้ และบอกว่า **บล็อกบอทแล้วข้อมูลยังอยู่** ต้องแจ้งทีมถ้าอยากให้ลบจริง

จุดที่พบบทอนทำ และควรเล่าให้กรรมการฟัง: ก่อนหน้านี้นี้หน้าตั้งค่ามีสวิตช์ "ผู้ช่วย AI" ที่ถูกล็อกไว้และแสดงค่าระดับระบบ ผู้ใช้ไม่มีทางถอนความยินยอมได้เลย ถ้าเราเขียนประกาศว่า "ปิดได้" โดยไม่แก้สวิตช์ ก็เท่ากับโฆษณาสิทธิที่ให้จริงไม่ได้

8. ฐานข้อมูล — 12 ตาราง

ไฟล์: `supabase/migrations/001_init.sql`

#	ตาราง	เก็บอะไร	จุดที่ต้องรู้
1	<code>users</code>	บัญชีผู้ใช้ LINE	<code>ai_enabled</code> · <code>daily_summary_enabled</code> · <code>email_ingest_token</code> · ไม่เก็บชื่อและรูปโปรไฟล์
2	<code>categories</code>	หมวดของแต่ละคน	<code>is_essential</code> ใช้ในสูตร capacity · ผู้ใช้ใหม่ได้ 11 หมวดเริ่มต้น
3	<code>plans</code>	แผนออกม	<code>status</code> : draft / active / done / cancelled

#	ตาราง	เก็บอะไร	จุดที่ต้องรู้
4	<code>recurring_rules</code>	รายการประจำ	
5	<code>transactions</code>	หัวใจของทั้งระบบ	ดูรายละเอียดด้านล่าง
6	<code>budgets</code>	งบรายเดือนต่อหมวด	เก็บธวงว่าเดือน 80%/100% ไปแล้วหรือยัง
7	<code>user_keyword_map</code>	พจนานุกรม L3	☐ ตารางมีแล้วแต่ยังไม่ได้อัดใช้
8	<code>pending_actions</code>	คำขอรอยืนยัน	หัวใจของ G2
9	<code>user_emails</code>	อีเมลที่ forward เข้ามา	เก็บ <code>body_redacted</code> ไม่ใช่เนื้อเต็ม · <code>unique (user_id, message_id)</code> กันซ้ำ
10	<code>webhook_events</code>	กัน event ซ้ำจาก LINE	LINE retry ได้ จึงต้องมี
11	<code>push_log</code>	บันทึกการส่ง push	ใช้เซ็คควตา 280/เดือน
12	<code>ai_usage_log</code>	log การเรียก AI	ไม่มีคอลัมน์เก็บ prompt หรือคำตอบ ตาม G5

8.1 ตาราง `transactions` — ข้อจำกัดที่ DB บังคับให้

```
amount    numeric(12,2) not null check (amount > 0 and amount <= 10000000)
type       text check (type in ('income','expense','transfer'))
check (type = 'transfer' or plan_id is null)    -- G7 ระดับ DB

create unique index uq_tx_ref on transactions (user_id, ref_number)
  where ref_number is not null and deleted_at is null;    -- S10 ระดับ DB
```

3 จุดที่ต้องอธิบายได้:

1. `check (amount > 0)` — G3 ถูกบังคับที่ DB ด้วย ไม่ใช่แค่ในโค้ด

แม้มีบั๊กในโค้ด DB ก็ยังปฏิเสธ

1. `check (type = 'transfer' or plan_id is null)` — G7 ระดับ DB

รายการที่ไม่ใช่ transfer จะผูกกับแผนไม่ได้เด็ดขาด

1. `unique index` บน `ref_number` — กันสลิป/อีเมลใบเดิมซ้ำถึงระดับ DB

ไม่ต้องเชื่อแค่การตรวจในโค้ด · `where deleted_at is null` ทำให้ลบแล้วจดใหม่ได้

1. ลบบแบบ soft delete (`deleted_at`) ไม่ใช่ลบจริง → กู้คืนได้ และคำสั่ง `ยกเลิก`

มีปุ่ม "เอากลับคืน" ให้

8.2 RLS — ความปลอดภัยที่ต้องอธิบายให้ถูก

```
alter table users enable row level security;    -- ... ทุกตาราง
-- แต่ "ไม่สร้าง policy ใดๆ เลย"
```

ผลลัพธ์: anon key / authenticated key เข้าตารางเหล่านี้ไม่ได้เลยแม้แต่แถวเดียว มีแค่ `service_role` key (ที่ backend ใช้) เท่านั้นที่ข้าม RLS ได้

สิ่งที่ต้องพูดให้ชัดถ้าถูกถามเรื่องความปลอดภัย:

RLS ของเราไม่ได้ทำหน้าที่แยกข้อมูลระหว่างผู้ใช้ — มันทำหน้าที่ปิดประตูไม่ให้ใคร เข้าถึง DB จากข้างนอกได้เลย การแยกข้อมูลให้ถูกคนเป็นความรับผิดชอบของโค้ด backend 100% นั่นคือเหตุผลที่ G6 สำคัญ และเป็นเหตุผลที่ทุกฟังก์ชันใน `db/queries/` มี `.eq('user_id', ...)`

9. หน้าเว็บ

9.1 6 หน้า

ไฟล์	หน้า	ทำอะไร
<code>index.html</code>	สรุป	ยอดคงเหลือ · รายรับ/รายจ่ายเดือนนี้ · รายการล่าสุด · ใช้ได้วันละเท่าไร
<code>transactions.html</code>	รายการ	ดู/เพิ่ม/แก้/ลบ
<code>analyze.html</code>	วิเคราะห์	กราฟโดนัทรายหมวด · กราฟแนวโน้ม 6 เดือน · จำลองก่อนซื้อ
<code>categories.html</code>	หมวดหมู่	จัดการหมวดและงบ
<code>settings.html</code>	ตั้งค่า	อีเมล forward · สวิตช์ปิด AI · ออกจากระบบ
<code>prototype.html</code>	prototype	เปิดดูได้โดยไม่ต้องมี server (ตามเกณฑ์ M3)

9.2 ทำไมมี `boot.js` — คำถามที่กรรมการน่าถาม

web/js/		
└─ <code>app.js</code>	(250 KB)	UI ทั้งหมด เขียนไว้ตอนยังใช้ mock data ล้วน ๆ
└─ <code>boot.js</code>	(52 KB)	ชั้นแปลง: ดึงจาก API จริง → เขียนทับ <code>window.mockData</code>
└─ <code>api.js</code>	(6.8 KB)	wrapper ของ <code>fetch</code>
└─ <code>mock-data.js</code>	(14 KB)	ข้อมูลตัวอย่างสำหรับโหมด preview
└─ <code>prototype-bar.js</code>	(7.8 KB)	แถบแจ้งเตือนในโหมด preview

คำตอบ: `app.js` ถูกเขียนตอนยังไม่มี backend โดยอ่านทุกอย่างจาก `window.mockData` การรีไทร์ไฟล์ 250 KB ทั้งไฟล์เพื่อให้รับ API เสียงเกินไปในเวลาที่มี เราจึงทำ **ชั้นแปลงข้อมูล** (`boot.js`) ที่ดึงจาก API จริงแล้วเขียนทับ `window.mockData` ในที่เดิม (ไม่ใช่แทนที่ตัวแปร เพราะ `app.js` จับ reference ไว้ตั้งแต่โหลด) แล้วสั่ง render ใหม่

นี่คือการตัดสินใจเชิงวิศวกรรมที่ควรเล่า — มันคือการเลือกระหว่าง "ทำให้สะอาด" กับ "ทำให้เสร็จและไม่พัง" ในรอบเวลาของวิชา และเรารู้ตัวว่าแลกอะไรไป (`app.js` ยังมีโค้ด mock ค้างอยู่ · `charts.js` เป็นซากที่ไม่ได้ใช้)

🛠️ G1 ในหน้าเว็บ: `boot.js` แปลง ข้อมูลเท่านั้น ห้ามคำนวณเงิน หักยอดมาจาก API ตรง ๆ ที่คำนวณได้มีแค่เรื่องแสดงผล (สีของหมวด ป้ายวันที่)

9.3 โหมด preview — ทำไมต้องมี

เกณฑ์ M3 บอกว่า prototype ต้อง "เปิดดูได้ในเบราว์เซอร์" แต่เว็บจริงของเรามีหน้า login กัน ถ้ากรรมการเปิดลิงก์แล้วเจอแต่หน้าล็อกอิน จะดูเหมือนไม่มีอะไร จึงมีปุ่ม "ดูตัวอย่างก่อน" ที่เข้าโหมด preview (`?preview=1`) ซึ่งแสดงข้อมูลตัวอย่าง พร้อมแถบแจ้งว่าเป็นข้อมูลสมมติ

9.4 Session — ล็อกอินด้วย LINE Login

```
/auth/login      → redirect ไป LINE
/auth/callback   → แลก code เป็น id_token → เซ็น session cookie → redirect เข้าเว็บ
/auth/logout     → ลบ cookie
```

cookie เซ็นด้วย HMAC จาก `SESSION_SECRET` และ payload ไม่ได้เข้ารหัส แคเซ็น → ใครเปิดดูก็เห็น user id ได้ จึงห้ามใส่ความลับลงไป (ห้ามใส่ token ของ LINE เด็ดขาด)

10. งานตามเวลา (cron)

Render free tier ไม่มี cron ให้ เราจึงใช้ GitHub Actions ยิง HTTP มาที่ `/jobs/run` พร้อม header `x-cron-secret`

งาน	ตาราง cron (UTC)	เวลาไทย	ทำอะไร
<code>emailPoll</code>	<code>* / 5 * * * *</code>	ทุก 5 นาที	ดึงอีเมลธนาคาร → แยกยอด → dedup → บันทึก
<code>recurring</code>	<code>0 23 * * *</code>	06:00	สร้างรายการประจำที่ถึงรอบ
<code>cleanup</code>	<code>0 20 * * *</code>	03:00	หมดอายุ pending · ยกเลิก draft เก่า · ลบ log เก่า
<code>planCheck</code>	<code>0 2 * * *</code>	09:00	เช็ควางแผนยังไหวไหม · ปิดแผนที่ครบเป้า
<code>dailySummary</code>	—	—	☐ ยังไม่ได้เขียน (FR-17)

ทุกงานต้อง idempotent — รันซ้ำแล้วต้องไม่เกิดผลซ้ำ เพราะ GitHub Actions รับประกันเวลาไม่ได้และอาจยิงซ้ำ:

- `emailPoll` กันซ้ำด้วย `Message-ID` + S10 dedup
- `recurring` กันซ้ำด้วย `unique (recurring_rule_id, recurring_run_date)` ใน DB

`cleanup` มีกฎเหล็กของตัวเอง: ๑ ห้ามแตะ `transactions` เด็ดขาด เงินของผู้ใช้ไม่มีวันหมดอายุ ตารางที่ลบได้คือ log และคำขอที่ค้างเท่านั้น

11. การทดสอบ — 683 ข้อ / 32 ไฟล์

11.1 หลักการ

ทดสอบว่า "ตัดสินใจถูกไหม" ไม่ใช่ "ต่อ DB ติดไหม"

ชั้น `db/queries/` ถูก mock ทั้งหมด เทสต์จึงรันจบใน ~7 วินาที และรันได้โดยไม่ต้องมี DB `SPEC.md` ยังบังคับว่า "ห้ามเรียก Gemini จริงใน test ต้อง mock เสมอ"

11.2 เทสต์ทั้งหมด

ไฟล์	จำนวน	พิสูจน์อะไร
------	-------	-------------

`aiTools.test.ts` | 45 | Zod + sanity ของ 8 tools — หลักฐานของ G1 | `aiRouter.test.ts` | 41 | ทางเดิน L4 ทั้งเส้น + เทสต์เชิงโครงสร้างว่าไม่มีใครเลี้ยง guard | `plan.service.test.ts` | 40 | สูตร capacity · 3 ทางเลือก · cold start | `recurring.service.test.ts` | 31 | รอบถัดไป · กรณี 31 ม.ค. | `money.test.ts` · `thaiNumber.test.ts` | 30 · 30 | หน่วยเงิน (G3) · อ่านเลขไทย | `pending.service.test.ts` | 30 | วงจร G2 · กดซ้ำ · คำนวณสถานะเมื่อเขียนล้ม | `thaiDateText.test.ts` | 29 | แปลงค่าบอกรวัน — เน้นทางที่ต้องคืน `null` | `kbankParser.test.ts` | 28 | แยกยอดจากอีเมล KBank | `regexParser.test.ts` · `budget.service.test.ts` | 27 · 27 | ทางด่วน L1 · เตือน 80/100% | `aiVision.test.ts` | 25 | ตรวจสอบ JSON ของสลิป — เน้นของที่ต้องไม่ผ่าน | `command.service.test.ts` | 25 | คำสั่งตายตัว | `imageHandler.test.ts` | 22 | วงจรอ่านสลิป + เทสต์ว่าไม่เขียนรูปลงดิสก์ | `thaiDate.test.ts` | 22 | วันที่ตามเวลาไทย | `transaction.service.test.ts` | 21 | CRUD รายการ | `simulate.service.test.ts` · `category.service.test.ts` | 18 · 18 | จำลองซื้อ · จัดการหมวด | `privacy.test.ts` · `session.test.ts` | 17 · 17 | PDPA · session cookie | `aiGuard.test.ts` · `redact.test.ts` | 16 · 16 | guard 5 ด้าน (G4) · ปิดข้อมูล (G5) | `emailDkim.test.ts` | 15 | ตรวจสอบ DKIM ของอีเมล | `planTransfer.test.ts` | 14 | โอนเข้าแผน (G7) | `safeToSpend.test.ts` | 13 | สูตรตอบคำถามที่ 1 | `planCheck.service.test.ts` · `summary.service.test.ts` | 12 · 12 | เช็คแผน · ยอดสรุป | `push.service.test.ts` | 11 | โควตา push | `dedup.service.test.ts` · `emailTokenHeaders.test.ts` | 9 · 9 | S10 · token อีเมล | `aiOptOut.test.ts` | 7 | ปิด AI รายคน | `cleanup.service.test.ts` | 6 | งานบำรุงรักษา |

11.3 แนวการเขียนเทสต์ที่ควรเล่า

เน้น "ทางที่ผิด" มากกว่า "ทางที่ถูก" เพราะความเสียหายไม่สมมาตร เช่น `thaiDateText.test.ts` มี 29 เทสต์ ซึ่งเกินครึ่งเป็นเคสที่ต้องคืน `null`: อ่านวันที่ผิดแล้วบันทึก = ยอดของวันนั้นเพี้ยนถาวรโดยผู้ใช้ไม่มีทางรู้ ส่วนอ่านไม่ออกแล้วถามกลับ = ผู้ใช้เสียเวลา 1 ข้อความ

เทสต์เชิงโครงสร้าง — บางเทสต์ตรวจสอบ "โครงสร้างโค้ด" ไม่ใช่พฤติกรรม:

- `aiRouter.test.ts` อ่านทุกไฟล์ใน `src/` ยืนยันว่ามีแค่ 3 ไฟล์ที่ import `@google/genai`
- `imageHandler.test.ts` อ่าน source ยืนยันว่าไม่มีคำว่า `writeFile` / `node:fs` / `/tmp`
- `privacy.test.ts` ยืนยันว่าประกาศมีคำว่า `free tier` และ `ปรับปรุงบริการ`

เทสต์แบบนี้บังคับ เจตนาเชิงออกแบบ ที่ปกติบังคับได้แค่ด้วยการ review

11.4 บั๊กที่เทสต์จับได้จริง (ตัวอย่างที่เล่าได้)

บั๊ก	ผลถ้าปล่อยไป
<code>new Date('2026-02-31T10:00:00+07:00')</code> → 3 มี.ค. ไม่ใช่ Invalid Date สลิปที่อ่านวันผิดถูกบันทึกเลื่อนไป 3 วันเฉยๆ <code>saveEntry</code> fix <code>source: 'chat'</code> ไว้ สลิปถูกบันทึกเป็น <code>chat</code> → คำสั่ง <code>ยกเลิก</code> ที่กรอง <code>source='chat'</code> จะไปลบรายการจากสลิปได้ <code>payload.occurredAtIso</code> รับแค่ <code>YYYY-MM-DD</code> เวลาสลิปถูกปิดเป็นเที่ยงวัน → สลิป 2 ใบยอดเท่ากันที่โอนหักกันหลายชั่วโมงถูกมองว่าซ้ำ <code>isoDayOfWeek</code> คืน 0-6 ไม่ใช่ 1-7 <code>"วันจันทร์"</code> แปลงเป็นวันผิด <code>redact</code> เดิมจับแต่เลขติดกันล้วน <code>"บัตร 1234 5678 9012"</code> ไม่ถูกปิด → เลขบัตรหลุดไปถึง Gemini	

12. Deploy — ระบบจริงทำงานอยู่ที่ไหน

ส่วน	ที่ไหน	จุดที่ต้องรู้
Backend + หน้าเว็บ	Render free tier · deploy จาก branch <code>backend</code>	หลับเมื่อไม่มีคนใช้ ~15 นาที ตื่นใช้ 30-60 วินาที

ส่วน	ที่ไหน	จุดที่ต้องรู้
ฐานข้อมูล	Supabase free tier	
งานตามเวลา	GitHub Actions	<code>.github/workflows/cron.yml</code>
AI	Google Gemini free tier · <code>gemini-3.5-flash-lite</code>	
ระบบอีเมล	Gmail + plus-address (<code>user+token@gmail.com</code>)	ตั้งด้วย IMAP

12.1 ตัวแปรสภาพแวดล้อมที่จำเป็น

LINE_CHANNEL_ACCESS_TOKEN · LINE_CHANNEL_SECRET	← บอท
LINE_LOGIN_CHANNEL_ID · LINE_LOGIN_CHANNEL_SECRET	← ล็อกอินเว็บ
SUPABASE_URL · SUPABASE_SERVICE_ROLE_KEY	← ฐานข้อมูล
SESSION_SECRET	← เซสชัน cookie
CRON_SECRET	← ป้องกัน /jobs
GEMINI_API_KEY	← AI (ไม่มีก็ยังไม่ทำงานได้ ตาม G4)
GMAIL_USER · GMAIL_APP_PASSWORD	← ระบบอีเมล
APP_BASE_URL	← ประกอบ redirect_uri

เปิด `/health` เพื่อดูว่าตั้งค่าครบไหม — คืน `notConfigured: []` ถ้าครบ นี่คือเครื่องมือ debug ที่ใช้บ่อยที่สุดตอน deploy

12.2 รันในเครื่อง

```
git clone https://github.com/6931503045-design/Lumen_line.git
cd Lumen_line
npm install
cp .env.example .env      # กรอกค่า LINE / Supabase
npm test                  # 683 tests
npm run dev                # http://localhost:3000
```

13. คำถามที่กรรมการน่าจะถาม + คำตอบ

ส่วนนี้สำคัญที่สุด — ซ้อมตอบด้วยปากจริง ๆ อย่างแผ่ว

Q1. "ระบบนี้ทำอะไร อธิบายใน 30 วินาที"

JOD tang เป็นบอท LINE ที่ช่วยนักศึกษาจัดเงินและวางแผนออม จุดต่างคือเราไม่ได้ทำแอป ที่บอกว่าเงินหายไปไหน แต่ตอบสองคำถามที่คนอยากรู้จริง: "วันนี้ใช้ได้อีกเท่าไร" กับ "จะเก็บเงินให้ถึงเป้าได้อย่างไร" จัดได้ 3 ทาง — พิมพ์ในแชท ส่งรูปสลิป หรือ forward อีเมลธนาคาร ส่วน AI ช่วยอ่านภาษาคนแต่ **ไม่คิดเลขเงินเลย** เพราะการคำนวณทุกบาทอยู่ในโค้ดที่เขียนทดสอบครอปได้

Q2. "ทำไมไม่ให้ AI คำนวณไปเลย จะง่ายกว่าไหม"

ง่ายกว่าตอนเขียน แต่ใช้จริงไม่ได้ครับ เพราะ AI ตอบด้วยน้ำเสียงมั่นใจเท่ากันทั้งตอนถูก และตอนผิด ผู้ใช้ไม่มีทางแยกได้ และเราก็เขียนทดสอบให้มันไม่ได้ เมื่อสูตรอยู่ในโค้ด เราเขียนทดสอบได้ว่า "ใส่ข้อมูลชุดนี้ ต้องได้ตัวเลขนี้เป๊ะ" ตอนนี้มีเทสต์ 683 ข้อ และ 13 ข้อในนั้นครอบคลุม safe-to-spend อย่างเดียว

Q3. "AI เข้าใจผิดแล้วบันทึกเงินผิดจะอย่างไร"

AI เขียนฐานข้อมูลเองไม่ได้เลยครับ มันเขียนได้แค่ "คำขอ" ลงตาราง `pending_actions` การเขียนจริงเกิดตอนผู้ใช้กดปุ่มยืนยันเท่านั้น และการรูดที่หักจะรอยอดที่จะบันทึกจริง ไม่ใช่ยอดที่ผู้ใช้พิมพ์ — เช่นกรณีหารบิล `ค่าข้าว 300 หารสามคน` การรูดจะโชว์ `฿100` พร้อมวงเล็บว่า `(หาร 3 คน จาก ฿300)` ถ้าโชว์ 300 แต่บันทึก 100 การกดยืนยันก็ไม่มีผล

Q4. "ถ้า AI ล่มหรือโศกนาฏกรรม แอปใช้ได้ไหม"

ใช้ได้ครับ เป็นกฎข้อ G4 ที่เราเขียนไว้ตั้งแต่ก่อนเขียนโค้ด ตั้ง `AI_ENABLED=false` แล้วทางด่วน regex (`กาแฟ 80`) คำสั่งทั้งหมด (สรุป เหลือ งบ แผน) ระบบอีเมล และหน้าเว็บยังทำงานเหมือนเดิม สิ่งที่ยหายไปคือการอ่านประโยคยาวกับอ่านสลิป เท่านั้น — และมีเทสต์ 16 ข้อใน `aiGuard.test.ts` ที่ยืนยันว่า guard ไม่ throw ไม่ว่าอะไรจะขึ้น เพราะถ้ามัน throw ข้อความของผู้ใช้จะพังทั้งข้อความ

Q5. "ข้อมูลของผู้ใช้คนหนึ่งจะหลุดไปหาคนอื่นได้ไหม"

เราเปิด RLS ทุกตาราง โดยไม่สร้าง policy เลย ผลคือ key ฝั่ง client เข้า DB ไม่ได้ แม้แต่แถวเดียว มีแค่ `service_role` key ที่ backend ใช้ที่ข้ามได้ แปลว่า การกรอ `user_id` ให้ถูกคนเป็นความรับผิดชอบของโค้ด 100% ไม่มีตาข่ายรองรับ นั่นคือเหตุผลที่เราตั้งเป็นกฎ G6 และทุกฟังก์ชันใน `db/queries/` มี `.eq('user_id', ...)` อีกจุดคือ `user_id` ไม่เคยรับมาจาก client — ฝั่งแซท query จาก `line_user_id` ของ event ฝั่งเว็บอ่านจาก session cookie ที่เซ็นด้วย HMAC

Q6. "เรื่อง PDPA จัดการยังไง ส่งข้อมูลไปให้ Google ด้วยไหม"

ส่งครับ และเราแจ้งผู้ใช้อย่างตรง ๆ สิ่งที่จะส่งออกไปคือข้อความที่ regex อ่านไม่ออกกับรูปสลิป ก่อนส่งมีการปิดเลข 9-19 หลัก (เลขบัญชี เบอร์โทร) ให้เป็น `[REDACTED]` ทุกครั้ง prompt ไม่มียอดเงินคงเหลือของผู้ใช้เลย มีแค่วันที่วันนี้ · รูปสลิปไม่ถูกเขียนลงดิสก์ ประวัติแชทเก็บใน memory 3 turn / 30 นาทีไม่ลง DB และตาราง `ai_usage_log` ไม่มีคอลัมน์เก็บข้อความเลย เราบอกผู้ใช้อย่างตรง ๆ ใช้ free tier ซึ่งตามเงื่อนไข Google อาจนำข้อมูลไปใช้ปรับปรุงบริการได้ และมีสวิตช์ให้ปิด AI เองในหน้าตั้งค่า ปิดแล้วยังเจตเงินได้ครับ

Q7. "ทำไมเงินเป็นสตางค์ ไม่เป็นบาททศนิยม"

เพราะ `0.1 + 0.2` ใน JavaScript ไม่เท่ากับ `0.3` ครับ ถ้าเก็บเงินเป็นทศนิยมแล้วบวกกันหลายพันครั้ง ยอดจะเพี้ยนทีละเศษสตางค์ เราจึงเก็บเป็นจำนวนเต็มสตางค์ และแปลงตอนแสดงผลเท่านั้น `utils/money.ts` ยังแปลงด้วย `string-parsing + BigInt` ไม่ใช่ `× 100` เพราะการคูณ 100 กับทศนิยมก็เพี้ยนได้เอง

Q8. "หารบิลกับเพื่อนอย่างไรให้เศษไม่หาย"

เราไม่ให้ AI หาร AI ส่งยอดเต็มกับจำนวนคนมา แล้วโค้ดหารด้วย `splitEvenly()` ซึ่งแจกเศษสตางค์ให้คนแรก ๆ ทีละ 1 สตางค์ `splitEvenly(10000, 3)` ได้ `[3334, 3333, 3333]` รวมได้ 10000 เป๊ะ ไม่ใช่ `3333 × 3 = 9999` ที่หายไป 1 สตางค์ และก่อนของผู้ใช้คือก่อนแรก (ก่อนที่ได้เศษ) เพื่อไม่ให้จบน้อยกว่าที่จ่ายจริง

Q9. "เจอปัญหาอะไรที่ไม่คาดคิดบ้าง" ← คำถามนี้ถามบ่อย ตอบให้ดี

เลือกเล่า 2-3 เรื่องจากนี้ ทุกเรื่องเกิดขึ้นจริงในโปรเจกต์:

1. Google ปิดรุ่นโมเดลที่เราใช้ — เราตั้ง default เป็น `gemini-2.0-flash` แล้ว Google ปิดรุ่นนั้นไป ผลคือใส่ API key ถูกต้องแล้วบอทก็ยังอ่านภาษาคนไม่ได้ แต่ แอปไม่พัง เพราะ G4 ทำงาน — guard ตีกลับแล้วตอบคำแนะนำทางด่วนให้อาการจึง "เจียบ" และหาสาเหตุยาก เราแก้โดยเปลี่ยนเป็น `gemini-3.5-flash-lite` และใส่คอมเมนต์เตือนไว้ในโค้ด ห้ามย้อนกลับ บทเรียน: การพึ่ง API ภายนอกคือหนี้ที่ต้องจ่ายคืนเมื่อเขาเปลี่ยน

2. ทางที่สำเร็จไม่มี log ทำให้แยกไม่ออกว่าจะอะไรเกิดขึ้น — ตอนแรกเรา log แคตอน error พอทดสอบแล้วไม่มีอะไรขึ้นใน log เลย เราแยกไม่ออกว่า "ไม่มีใครส่งข้อความมา" หรือ "ทำงานได้ปกติ" เราจึงเพิ่ม log ตอนสำเร็จที่ลงแค่ชื่อ tool กับ เวลาที่ใช้ ไม่มีข้อความหรือยอดเงินของผู้ใช้ ตาม G5

3. JavaScript ปิดวันที่ที่ไม่มีอยู่จริงให้เองเจียบ ๆ — `new Date('2026-02-31...')` ได้ 3 มีนาคม ไม่ใช่ Invalid Date ถ้าไม่ดัก สลิปที่ AI อ่านวันผิดจะถูกบันทึกเลื่อนไป 3 วัน โดยไม่มีใครรู้ เทสต์จับได้ตอนเขียนพีเจอรอ่านสลิป

4. บั๊ก Gmail plus-address — ไม่มีทางเจอจาก unit test เพราะ fixture ไม่มี header ของจริง ต้องส่งอีเมลจริงมาทดสอบ บทเรียน: ต้องทดสอบกับของจริงเสมอ

Q10. "AI เขียนโค้ดให้เท่าไร ส่วนไหนเป็นของทีม"

ตอบตามตรงและอย่าเลี้ยง — เกณฯวิชานี้ไม่ได้ห้ามใช้ AI แต่ให้คะแนนความเข้าใจ

AI ช่วยเขียนโค้ดเยอะครับ แต่สิ่งที่เป็นของทีมคือ การตัดสินใจทั้งหมด — กฎเหล็ก 7 ข้อ สูตรคำนวณ การจัดชั้นโค้ด และการเลือกว่าอะไรเป็น Must อะไรเป็น Could เราเขียน `AIDO.md` 750 บรรทัดเป็นคู่มือกำกับ AI ตั้งแต่ก่อนเขียนโค้ด โดยระบุ "เขตห้าม AI" ไว้ชัดเจน — Money Engine กับ Dedup ห้าม AIแตะเด็ดขาด บทเรียนที่ชัดที่สุดคือ AI เร่งการเขียนโค้ดได้ แต่ไม่ได้เร่งการ "รู้ว่าคุณ" เทสต์ 683 ข้อคือสิ่งที่ทำให้เรากล้าแก้โค้ด ไม่ใช่ความมั่นใจของ AI

Q11. "มีอะไรที่ยังทำไม่เสร็จ"

ตอบตรง ๆ ดีกว่าถูกจับได้ · ดู §15

เทียบกับ SRS ที่ส่ง M2 แล้วครบ 15/15 ข้อครับ ส่วนในฉบับไทยที่ละเอียดกว่า (20 FR) เหลือ 2 ข้อซึ่งเป็น **Could** ทั้งคู่ — สรุปรายวันส่งเข้า LINE กับ Rich Menu ทั้งสองข้อตัดออกโดยเจตนาตาม MoSCoW ที่เราวางไว้แต่ต้น ไม่ใช่ทำไมทัน เพราะการตัดออกไม่กระทบคำถามหลักสองข้อที่โปรเจกต์ตั้งใจจะตอบ

Q12. "ทำไมต้องมี 4 ชั้นใน router ซับซ้อนไปไหม"

ไม่ครับ มันลดความซับซ้อน เพราะแต่ละชั้นแก้ปัญหต่างกัน `กาแฟ 80` ให้ regex อ่านถูก 100% ฟรี และทันที ส่วน AI อ่านถูก "เกือบ" 100% มีค่าใช้จ่าย และรอหลายวินาที การส่งทุกข้อความไปให้ AI คือการจ่ายแพงกว่า เพื่อผลที่แม่นยำกว่าในเคสที่พบบ่อยที่สุด — และถ้า AI ล่ม ชั้น 1-2 ยังทำงาน

Q13. "ถ้ามีเวลาอีก 1 เดือน จะทำอะไร"

เลือก 2-3 ข้อแล้วให้เหตุผล

1. เขียน L2/L3 พจนานุกรมหมวด — ตารางมีแล้ว (`user_keyword_map`) จะลดการเรียก AI ลงมากเพราะระบบจะจำได้ว่าผู้ใช้คนนี้เรียก "ชาเย็น" ว่าหมวดอาหาร 2. Flex card 4 ใบ — ตอนนี้คำตอบในแชทเป็นข้อความล้วน ใช้งานได้ครบแต่หน้าตายังไม่เท่ 3. หน้าที่ให้ผู้ใช้งานลบข้อมูลตัวเองได้ — ตอนนี้ต้องแจ้งทีม ซึ่งเป็นสิทธิตาม PDPA ที่ควรทำเป็นฟีเจอร์ ไม่ใช่กระบวนการ manual

14. ตัวเลขและคัพท์ที่ต้องจำได้

14.1 ค่าคงที่ (`src/config/constants.ts`)

ค่า	ตัวเลข	ความหมาย
<code>PUSH_LIMIT</code>	280	โควตา push ของ LINE ต่อเดือน (free tier)
<code>MAX_ACTIVE_PLANS</code>	3	แผนออมที่เปิดพร้อมกันได้สูงสุดต่อคน
<code>PLAN_SAFETY_RATIO</code>	0.8	กินพื้นที่เผื่อไว้ 20% ตอนคำนวณ capacity
<code>EMERGENCY_BUFFER</code>	0.15	กันเงินฉุกเฉิน 15% ของรายจ่ายเฉลี่ย
<code>MAX_AMOUNT_SATANG</code>	1,000,000,000	= 10 ล้านบาท เพดานต่อรายการ
<code>PENDING_EXPIRE_HOURS</code>	24	ค่าขอรอยืนยันหมดอายุ
<code>DEDUP_WINDOW_MINUTES</code>	30	หน้าต่างเวลาที่ถือว่าซ้ำ
<code>CHAT_HISTORY_TURNS</code>	3	ประวัติแชทที่ส่งให้ AI
<code>COLD_START</code>	7 / 90	วัน เกณฑ์ confidence low / medium
<code>BALANCED_RATIO</code> · <code>RELAXED_RATIO</code>	0.6 · 0.35	สัดส่วนทางเลือก สมดุล / สบาย
<code>LOOKBACK_DAYS</code>	90	ช่วงข้อมูลย้อนหลังที่ใช้คำนวณ capacity
<code>AI_DAILY_LIMIT_PER_USER</code>	30	โควตา AI ต่อคนต่อวัน
<code>AI_TIMEOUT_MS</code>	8000	รอ AI ไม่เกิน 8 วินาที

14.2 ตัวเลขของโปรเจกต์

เทสต์	683 ข้อ · 32 ไฟล์
ตารางในฐานข้อมูล	12
ไฟล์ TypeScript ใน <code>src/</code>	79
API endpoint	28 ได้ <code>/api</code>
หน้าเว็บ	6
งานตามเวลา	4 (ทำงานอยู่)
FR	15/15 ตาม PDF ที่ส่ง M2 · 18/20 ตามฉบับไทย
กฎหลัก	7 ข้อ
หมวดเริ่มต้น	11 หมวด

14.3 คัพท์ที่ใช้ในทีม

คัพท์	หมายถึง
safe-to-spend	"ใช้ได้อีกวันละเท่าไร" — ตัวเลขตอบคำถามที่ 1
capacity	เงินที่ออมได้ต่อเดือนโดยไม่พัง
disposable	รายรับ – ประจำ – จำเป็น – เงินกันฉุกเฉิน

ศัพท์	หมายถึง
cold start	ผู้ใช้ใหม่ที่ข้อมูลยังน้อยเกินกว่าจะเชื่อค่าเฉลี่ย
pending action	คำขอที่ AI สร้างไว้รอผู้ใช้กดยืนยัน
dedup	การกันรายการซ้ำข้ามช่องทาง
L1 / L4	ชั้นของ router — L1 = regex, L4 = AI
G1-G7	กฎเหล็ก 7 ข้อ
S1-S13	หัวข้อสถาปัตยกรรมใน SPEC.md
satang	หน่วยเงินที่ระบบใช้ภายใน (100 สตางค์ = 1 บาท)

15. ข้อจำกัดที่เรารู้ตัว — ตอบตรง ๆ ดีกว่าถูกจับได้

ข้อจำกัด	สถานะ · เหตุผล
FR-17 สรุปรายวันเข้า LINE	☐ <code>jobs/dailySummary.ts</code> เป็นไฟล์ว่าง · Could ตัดได้ตาม MoSCoW
FR-18 Rich Menu	☐ <code>line/richmenu.ts</code> เป็นไฟล์ว่าง · Could
Flex card 4 ใบ	☐ <code>summaryCard</code> <code>planCard</code> <code>errorCard</code> <code>pendingCard</code> ยังว่าง → คำตอบในแชทเป็นข้อความล้วนใช้งานได้ครบ แต่หน้าตาไม่ตรงที่ SPEC §S1 ชาติไว้
L2/L3 พจนานุกรมหมวด	☐ <code>keyword.service.ts</code> ว่าง · ตารางมีแล้ว · ตอนนี้จับคู่กับหมวดที่ผู้ใช้มีอยู่แทน
<code>web/js/charts.js</code>	 ซากที่ไม่ได้ใช้ (กราฟวาดใน <code>app.js</code>) ควรลบ
<code>app.js</code> ยังมีโค้ด mock ด้าง	ผลจากการเลือกทำชั้น <code>boot.js</code> แทนการรื้อทั้งไฟล์ — รู้ตัวและยอมรับ
Render กลับ	free tier · ข้อความแรกหลังกลับอาจไม่มีคำตอบเพราะ reply token หมดอายุก่อน
ลบข้อมูลตัวเองไม่ได้	ต้องแจ้งทีม · ควรทำเป็นฟีเจอร์ (ลืมหิดตาม PDPA)
ธนาคารที่อ่านอีเมลได้	มีแค่ KBank · เพิ่มธนาคารอื่นต้องเขียน parser ใหม่
SRS 2 ฉบับเลข FR ไม่ตรงกัน	ฉบับไทย 20 FR · PDF ที่ส่ง 15 FR — ถ้าถูกถามให้ยึดเลขตาม PDF
SRS เขียนว่าหมวดเริ่มต้น "12 หมวด" แต่โค้ดมี 11	<code>docs/SRS.md</code> บรรทัด 153 ไม่ตรงกับ <code>DEFAULT_CATEGORIES</code> ใน <code>constants.ts</code> (8 รายจ่าย + 3 รายรับ = 11) · กรรมการนับจากหน้าหมวดหมู่แล้วจับได้ง่าย ถ้าถูกถามให้ตอบ 11 แล้วบอกว่าเอกสารเขียนคลาดไป

16. เช็กลิสต์ก่อนเข้าห้องสอบ

- ☐ ทุกคนตอบ Q1 (ระบบทำอะไรใน 30 วินาที) ได้ด้วยปากตัวเอง
- ☐ ทุกคนอธิบาย กฎเหล็ก 3 ข้อ ได้ว่าทำไมต้องมี (แนะ G1 · G2 · G4)

- ☐ ทุกคนชี้ได้ว่า สูตร **safe-to-spend** อยู่ไฟล์ไหน และห้ทำอะไรออกบ้าง
- ☐ มีคนตอบ **Q9** (ปัญหาที่ไม่คาดคิด) ได้อย่างน้อย 2 เรื่อง
- ☐ มีคนตอบ **Q10** (AI ช่วยแค่ไหน) ได้โดยไม่เสีย
- ☐ ทุกคนรู้ว่า **ตัวเองเขียนไฟล์ไหน** และอธิบายไฟล์นั้นได้
- ☐ เปิด **/health** ได้และรู้ว่ามันบอกอะไร
- ☐ **ปลุก Render ก่อนเดโม** — เปิดเร็วไว้ 1 นาทีก่อนเริ่ม ไม่จ้นข้อความแรกจะเจียบ
- ☐ เตรียมสลิปจริง 1 ใบไว้เดโมพีเจอร์อ่านสลิป
- ☐ รู้ข้อจำกัดใน **S15** และพูดเองก่อนถูกถาม

เอกสารนี้สำหรับทีม Lumen อ่านเอง ไม่ใช่เอกสารส่ง · ถ้าแก้โค้ดที่กระทบสูตรหรือกฎเหล็ก ให้กลับมาแก้ไฟล์นี้คู่กัน ไม่จ้นเอกสารจะกลายเป็นข้อมูลเท็จซึ่งแย่กว่าไม่มีเอกสาร